

Spring Interview Questions by Java Scaler

Easy Level Questions for Spring

1. What is Spring Framework?

- The Spring Framework is an open-source application framework for Java. It provides comprehensive infrastructure support for developing Java applications, allowing developers to focus on business logic rather than boilerplate code.

2. What are the main features of Spring Framework?

- The main features of Spring Framework include:
 - Inversion of Control (IoC)
 - Aspect-Oriented Programming (AOP)
 - Transaction management
 - Data access and persistence
 - MVC framework for web applications

3. What is Inversion of Control (IoC) in Spring?

- Inversion of Control (IoC) is a design principle where the control of object creation and dependency management is inverted from the application code to the Spring container. This helps in achieving loose coupling and easier testing.

4. What is a Spring Bean?

- A Spring Bean is an object that is instantiated, assembled, and managed by the Spring IoC container. Beans are defined in the Spring configuration file or by using annotations.

5. How do you define a Spring Bean in XML configuration?

```
<bean id="myBean" class="com.example.MyBean"/>
```

- Here, `id` is the name of the bean and `class` specifies the fully qualified class name of the bean.

6. What are the different types of Spring Bean scopes?

- The different types of Spring Bean scopes are:
 - `singleton`
 - `prototype`
 - `request`
 - `session`
 - `global-session`

7. What is Dependency Injection (DI) in Spring?

- Dependency Injection (DI) is a design pattern used to implement IoC. It allows an object's dependencies to be injected by the container at runtime, promoting loose coupling and easier testing.

8. What are the different ways to perform Dependency Injection in Spring?

- The different ways to perform Dependency Injection in Spring are:
 - Constructor Injection
 - Setter Injection
 - Field Injection (using annotations)

9. What is the difference between Constructor Injection and Setter Injection?

- Constructor Injection provides dependencies through a class constructor, while Setter Injection provides dependencies through setter methods. Constructor Injection is preferred for mandatory dependencies, while Setter Injection is used for optional dependencies.

10. How do you configure a bean using annotations in Spring?

```
@Component
public class MyBean {
    // class body
}
```

- Here, `@Component` annotation indicates that this class is a Spring bean.

11. What is `@Autowired` annotation in Spring?

- The `@Autowired` annotation is used for automatic dependency injection in Spring. It can be applied to fields, setters, and constructors.

12. What is `@Component` annotation in Spring?

- The `@Component` annotation indicates that a class is a Spring component. It is a generic stereotype annotation for any Spring-managed component.

13. Explain the use of `@Service` annotation in Spring.

- The `@Service` annotation is a specialization of the `@Component` annotation, indicating that an annotated class is a service layer component in the Spring application.

14. What is the `@Repository` annotation in Spring?

- The `@Repository` annotation is a specialization of the `@Component` annotation, indicating that an annotated class is a DAO (Data Access Object) component. It also provides automatic exception translation.

15. What is the `@Controller` annotation in Spring?

- The `@Controller` annotation is a specialization of the `@Component` annotation, indicating that an annotated class is a web controller in the Spring MVC framework.

16. What is the Spring Boot?

- Spring Boot is an extension of the Spring Framework that simplifies the development of Spring applications. It provides pre-configured templates and defaults, reducing the need for boilerplate configuration.

17. What are the advantages of using Spring Boot?

- The advantages of using Spring Boot include:
 - Simplified configuration
 - Embedded servers
 - Production-ready features
 - Easy integration with other Spring projects
 - Faster development and deployment

18. What is a Spring Boot Starter?

- A Spring Boot Starter is a pre-configured dependency descriptor used to simplify the addition of dependencies to a Spring Boot project. For example, `spring-boot-starter-web` for web applications.

19. How do you create a Spring Boot application?

- You can create a Spring Boot application using Spring Initializr, which provides a web interface to generate a Spring Boot project with the necessary dependencies and configurations.

20. What is the purpose of `application.properties` file in Spring Boot?

- The `application.properties` file is used to configure application properties and settings in a Spring Boot project. It allows customization of various aspects of the application.

21. What is Spring Data JPA?

- Spring Data JPA is a part of the Spring Data project, which provides an abstraction layer over JPA (Java Persistence API). It simplifies database access and reduces the amount of boilerplate code required.

22. How do you define a repository interface in Spring Data JPA?

```
@Repository
public interface UserRepository extends JpaRepository<User,
Long> {
    // custom query methods
}
```

- Here, `JpaRepository` provides CRUD operations and `UserRepository` can include custom query methods.

1. What is Spring AOP?

- Spring AOP (Aspect-Oriented Programming) allows you to modularize cross-cutting concerns (such as logging, security, transaction management) into separate aspects.

2. What is a Pointcut in Spring AOP?

- A Pointcut is an expression that matches join points (where advice should be applied). It defines the "where" part of AOP.

3. What is Advice in Spring AOP?

- Advice is the action taken by an aspect at a particular join point. It defines the "what" part of AOP and can be applied before, after, or around method execution.

4. Explain the use of `@Aspect` annotation in Spring.

- The `@Aspect` annotation is used to define a class as an aspect in Spring AOP. This class contains advice methods that are applied to specified join points.

5. What is Spring MVC?

- Spring MVC is a web framework within the Spring Framework that follows the Model-View-Controller (MVC) design pattern, helping to build web applications in a structured way.

6. What is a `DispatcherServlet` in Spring MVC?

- The `DispatcherServlet` is the central servlet that dispatches requests to appropriate handlers, controllers, and views in a Spring MVC application.

7. How do you handle exceptions in Spring MVC?

- Exceptions in Spring MVC can be handled using `@ExceptionHandler` methods, `@ControllerAdvice` classes, and global exception handling configurations.

8. What is the purpose of `@RequestMapping` annotation in Spring MVC?

- The `@RequestMapping` annotation is used to map web requests to specific handler methods or classes in Spring MVC controllers.

Medium to Hard Level Questions for Spring

1. How do you enable transaction management in Spring?

- Transaction management can be enabled using the `@EnableTransactionManagement` annotation and configuring a transaction manager bean.

2. What is the difference between `@Transactional` and programmatic transaction management in Spring?

- `@Transactional` annotation declaratively manages transactions, simplifying code by applying transaction boundaries around methods.

Programmatic transaction management involves explicit transaction handling in code, using `TransactionTemplate` or `PlatformTransactionManager`.

3. Explain the Spring Bean lifecycle.

- The Spring Bean lifecycle includes:
 1. Bean instantiation.
 2. Dependency injection.
 3. `BeanNameAware`, `BeanFactoryAware` callbacks.
 4. `@PostConstruct` annotated method.
 5. `InitializingBean.afterPropertiesSet()` method.
 6. Custom init method.
 7. Bean is ready for use.
 8. `@PreDestroy` annotated method.
 9. `DisposableBean.destroy()` method.
 10. Custom destroy method.

4. What is the difference between `ApplicationContext` and `BeanFactory`?

- `BeanFactory` is a basic container providing basic DI capabilities. `ApplicationContext` extends `BeanFactory` with additional features like event propagation, declarative mechanisms to create a bean, and easier integration with Spring's AOP and transaction management.

5. How do you define a custom scope for a Spring bean?

- Custom scopes can be defined by implementing the `Scope` interface and registering it using `ConfigurableBeanFactory.registerScope()`.

6. What is the purpose of the `@Bean` annotation in Spring?

- The `@Bean` annotation is used to declare a bean method in a Java configuration class, which will return an object that Spring manages as a bean.

7. How do you handle circular dependencies in Spring?

- Circular dependencies can be handled by using setter injection instead of constructor injection, or by using the `@Lazy` annotation to delay bean initialization.

8. Explain the use of `@EnableAutoConfiguration` annotation in Spring Boot.

- The `@EnableAutoConfiguration` annotation enables Spring Boot to automatically configure your application based on the dependencies present in the classpath.

9. What is Spring Security?

- Spring Security is a framework that provides comprehensive security services for Java applications, including authentication, authorization, and protection against common attacks.

10. How do you secure a Spring application using Spring Security?

Spring Security can be configured using `SecurityConfigurerAdapter` to set up authentication, authorization rules, and security filters.

11. What is the difference between `@PreAuthorize` and `@Secured` annotations?

- `@PreAuthorize` is more powerful and flexible, allowing SpEL (Spring Expression Language) expressions for more complex security rules.
- `@Secured` is simpler and only supports role-based access control.

12. How do you implement method-level security in Spring?

- Method-level security can be implemented using annotations like `@Secured`, `@PreAuthorize`, and `@PostAuthorize` on service layer methods.

13. What is the `RestTemplate` in Spring?

- `RestTemplate` is a synchronous client to perform HTTP requests in Spring, supporting CRUD operations, exchange of data, and more.

14. How do you create a RESTful web service in Spring?

```
@RestController
@RequestMapping("/api")
public class MyRestController {
    @GetMapping("/resource")
    public ResponseEntity<String> getResource() {
        return new ResponseEntity<>("Resource content",
        HttpStatus.OK);
    }
}
```

15. What is the `WebClient` in Spring WebFlux?

- `WebClient` is a non-blocking, reactive client for performing HTTP requests in Spring WebFlux.

16. Explain the concept of Reactive Programming in Spring.

- Reactive Programming in Spring is about developing non-blocking, asynchronous applications using reactive streams, which improve resource utilization and scalability.

17. What is Spring Cloud?

- Spring Cloud provides tools for building and deploying microservices-based architectures, handling configuration management, service discovery, circuit breakers, and more.

18. How do you implement service discovery in Spring Cloud?

- Service discovery can be implemented using Spring Cloud Netflix Eureka or Consul. Services register themselves and discover other services through a registry.

19. What is a Circuit Breaker in Spring Cloud?

- A Circuit Breaker pattern, implemented by Spring Cloud Circuit Breaker or Hystrix, helps prevent cascading failures in microservices by stopping the flow of requests to a failing service.

20. How do you configure load balancing in Spring Cloud?

- Load balancing can be configured using Spring Cloud Ribbon, which provides client-side load balancing.

21. What is the use of `@HystrixCommand` annotation?

- The `@HystrixCommand` annotation is used to define a method that will be executed with circuit breaker functionality, providing fallback methods in case of failures.

22. What are Spring Cloud Config Server and Config Client?

- Spring Cloud Config Server provides a centralized configuration service for distributed systems. Config Client fetches configuration properties from the Config Server to keep applications in sync with configuration changes.

23. How do you handle configuration management in Spring Cloud?

- Configuration management is handled using Spring Cloud Config Server, which stores configurations in a version-controlled repository like Git. Applications fetch configuration properties at startup or runtime.

24. What is Spring Batch?

- Spring Batch is a framework for developing robust batch processing applications. It supports large-scale processing of data sets, transaction management, job scheduling, and more.

25. How do you configure a Spring Batch job?

```
@Configuration
public class BatchConfig {
    @Bean
    public Job job(JobBuilderFactory jobBuilderFactory,
Step step) {
        return jobBuilderFactory.get("job")
            .start(step)
            .build();
    }

    @Bean
    public Step step(StepBuilderFactory stepBuilderFactory,
ItemReader<String> reader,
ItemProcessor<String, String> processor,
ItemWriter<String> writer) {
        return stepBuilderFactory.get("step")
            .<String, String>chunk(10)
            .reader(reader)
            .processor(processor)
            .writer(writer)
            .build();
    }
}
```

26. What is Spring Integration?

- Spring Integration is a framework for enterprise application integration, providing support for messaging, transformation, routing, and

integration with external systems.

27. How do you handle file uploads in Spring MVC?

```
@Controller
public class FileUploadController {
    @PostMapping("/upload")
    public String handleFileUpload(@RequestParam("file")
    MultipartFile file) {
        // process file
        return "uploadSuccess";
    }
}
```

28. What is a Spring Boot actuator?

- Spring Boot Actuator provides production-ready features such as monitoring, metrics, health checks, and application management endpoints.

29. How do you monitor a Spring Boot application?

- Monitoring a Spring Boot application can be done using Actuator endpoints, which provide information about application health, metrics, and environment. Tools like Prometheus and Grafana can be integrated for more advanced monitoring.

30. Explain the concept of auto-configuration in Spring Boot.

- Auto-configuration in Spring Boot automatically configures application components based on the dependencies present in the classpath, simplifying the setup and reducing boilerplate configuration.

Hard to Very Hard Level Questions for Spring

1. Explain the internal working of Spring IoC container.

- The Spring IoC container uses Dependency Injection to manage the lifecycle and dependencies of beans. It reads configuration metadata, instantiates beans, injects dependencies, and manages the entire bean lifecycle.

2. How does Spring manage transactions internally?

- Spring manages transactions using AOP proxies, which intercept method calls and manage transactions according to the specified transaction attributes. The `TransactionInterceptor` handles transaction boundaries and commits or rollbacks transactions.

3. What is Spring's Resource abstraction?

- Spring's Resource abstraction provides a unified way to access different types of resources, such as files, classpath resources, and URLs. The `Resource` interface and its implementations like `FileSystemResource` and `ClassPathResource` are used.

4. How do you create a custom Spring Boot starter?

- To create a custom Spring Boot starter, define the starter's dependencies in a `starter` module, create an auto-configuration class in an `autoconfigure` module, and use `spring.factories` to register the auto-configuration class.

5. Explain the architecture of Spring Cloud Stream.

- Spring Cloud Stream provides a framework for building event-driven microservices. It uses binders to connect to external messaging systems, defines inputs and outputs through annotations, and handles message serialization and deserialization.

6. What is the role of `@EnableScheduling` annotation in Spring?

- The `@EnableScheduling` annotation enables Spring's scheduled task execution capability, allowing methods annotated with `@Scheduled` to be executed at specified intervals.

7. How do you implement distributed tracing in a Spring application?

- Distributed tracing can be implemented using Spring Cloud Sleuth, which adds trace and span IDs to requests, logs, and integrates with tracing systems like Zipkin or Jaeger for visualization.

8. What is the role of `@EventListener` annotation in Spring?

- The `@EventListener` annotation is used to define a method as an event listener in Spring, allowing it to handle application events published by the `ApplicationEventPublisher`.

9. How do you customize Spring Boot Actuator endpoints?

- Spring Boot Actuator endpoints can be customized by implementing `Endpoint` interfaces, adding custom health indicators, and configuring properties in `application.properties` or `application.yml`.

10. What is the purpose of `@Conditional` annotation in Spring?

- The `@Conditional` annotation is used to conditionally include beans based on specified conditions. It allows fine-grained control over bean creation based on environment, classpath, or custom logic.

11. How do you implement caching in a Spring application?

- Caching in a Spring application can be implemented using the `@EnableCaching` annotation and cache annotations like `@Cacheable`, `@CachePut`, and `@CacheEvict`. Spring provides integration with various cache providers like EhCache, Redis, and Hazelcast.

12. Explain the internals of Spring's AOP implementation.

- Spring AOP is implemented using proxies (JDK dynamic proxies for interfaces and CGLIB proxies for classes). The `ProxyFactory` and `AdvisedSupport` classes manage the creation of proxies and weaving of advice at runtime.

13. How does Spring handle asynchronous processing?

- Asynchronous processing in Spring is handled using the `@Async` annotation on methods, along with `@EnableAsync` to enable asynchronous support. This allows methods to run in separate threads, improving application responsiveness.

14. What is Spring WebFlux and how does it differ from Spring MVC?

- Spring WebFlux is a reactive web framework introduced in Spring 5, designed for building non-blocking, asynchronous web applications. It differs from Spring MVC by using reactive programming models and supporting reactive streams.

15. How do you handle backpressure in a Spring WebFlux application?

- Backpressure in a Spring WebFlux application is handled using reactive streams' mechanisms, such as controlling data flow with `Publisher` and `Subscriber` interfaces, and using operators like `onBackpressureBuffer`, `onBackpressureDrop`, and `onBackpressureLatest`.

16. What is the purpose of `@SpringBootApplication` annotation?

- The `@SpringBootApplication` annotation is a convenience annotation that combines `@EnableAutoConfiguration`, `@ComponentScan`, and `@Configuration`. It marks the main class of a Spring Boot application.

17. How do you secure a REST API using OAuth2 in Spring?

- Securing a REST API using OAuth2 in Spring involves configuring an authorization server and a resource server, using Spring Security OAuth2 to handle token issuance, validation, and securing endpoints with appropriate scopes and roles.

18. Explain the internals of Spring's transaction management using AOP.

- Spring's transaction management uses AOP proxies to intercept method calls. The `TransactionInterceptor` applies transaction attributes, managing transaction boundaries, and coordinating with the underlying transaction manager to commit or rollback transactions.

19. What is the role of `@Profile` annotation in Spring?

- The `@Profile` annotation is used to indicate that a component should be included in the application context only when a specified profile is active. It allows configuring beans and components for different environments (e.g., dev, test, prod).

20. How do you handle concurrency in Spring applications?

- Concurrency in Spring applications is handled using task executors, `@Async` for asynchronous methods, and `@Scheduled` for scheduled tasks. Spring also provides support for managing thread pools and handling synchronization.

21. What is the difference between `@ComponentScan` and `@EntityScan` ?

- `@ComponentScan` is used to specify the base packages to scan for Spring components like `@Component`, `@Service`, and `@Repository`. `@EntityScan` is used to specify the packages to scan for JPA entities.

22. How do you implement custom validation in Spring?

- Custom validation in Spring can be implemented by creating custom validator classes that implement the `ConstraintValidator` interface and annotating model fields with the custom constraint annotations.

23. Explain the use of `@ConfigurationProperties` in Spring Boot.

- The `@ConfigurationProperties` annotation is used to bind external configuration properties to a Java object, allowing type-safe configuration and easy management of complex configuration properties.

24. What is the Spring Cloud Gateway?

- Spring Cloud Gateway is a library for building API gateways on top of Spring Boot and Spring WebFlux. It provides routing, filtering, and load balancing capabilities for microservices.

25. How do you integrate Spring with a messaging system like RabbitMQ or Kafka?

- Integration with messaging systems like RabbitMQ or Kafka can be done using Spring AMQP for RabbitMQ and Spring Kafka for Kafka. These libraries provide templates and listener containers for sending and receiving messages.

Questions for 3-5 Years of Experienced Person

1. How do you design a microservice architecture using Spring Boot and Spring Cloud?

- Designing a microservice architecture involves defining service boundaries, using Spring Boot for creating microservices, and Spring Cloud for service discovery, configuration management, circuit breakers, and messaging.

2. How do you implement event-driven microservices using Spring?

- Event-driven microservices can be implemented using Spring Cloud Stream to publish and consume events, configuring message channels, and handling message serialization and deserialization.

3. What are the best practices for securing Spring Boot applications?

- Best practices for securing Spring Boot applications include using HTTPS, securing sensitive endpoints, applying role-based access control, using Spring Security for authentication and authorization, and regularly updating dependencies to patch vulnerabilities.

4. How do you optimize the performance of a Spring application?

- Performance optimization can be done by using caching, optimizing database queries, using connection pooling, configuring proper thread

pools, and profiling the application to identify and address bottlenecks.

5. Explain the concept of API Gateway in a microservices architecture.

- An API Gateway is a server that acts as an API front-end, handling requests, routing them to appropriate microservices, performing authentication, rate limiting, and aggregating responses. Spring Cloud Gateway can be used to implement this pattern.

6. How do you handle database migrations in a Spring Boot application?

- Database migrations can be handled using tools like Flyway or Liquibase, which manage versioned scripts and automate the database schema changes across different environments.

7. What are the challenges of deploying Spring Boot microservices and how do you overcome them?

- Challenges include managing configurations, ensuring service discovery, handling inter-service communication, managing data consistency, and monitoring. Using Spring Cloud Config, Eureka, Feign clients, distributed transactions, and Spring Boot Actuator can help overcome these challenges.

8. How do you implement logging in a Spring Boot application?

- Logging can be implemented using SLF4J with Logback or Log4j2. Configure log levels, appenders, and formatters in `logback-spring.xml` or `log4j2-spring.xml` and use `@Slf4j` for logging in code.

9. How do you handle exceptions globally in a Spring Boot application?

- Global exception handling can be implemented using `@ControllerAdvice` along with `@ExceptionHandler` methods to handle specific exceptions and return appropriate responses.

10. How do you ensure data consistency in a distributed Spring application?

- Ensuring data consistency can be achieved using distributed transactions (e.g., using Spring Data JPA and JTA), eventual consistency patterns, and message queues for reliable event propagation.

11. How do you manage configurations in a Spring Cloud application?

- Configurations in a Spring Cloud application can be managed using Spring Cloud Config Server, which externalizes configuration properties

and serves them to client applications based on their environment.

12. What is a `HandlerInterceptor` in Spring MVC and how do you use it?

- A `HandlerInterceptor` is used to intercept requests in Spring MVC for pre-processing and post-processing. It can be implemented by extending the `HandlerInterceptorAdapter` class and overriding methods like `preHandle`, `postHandle`, and `afterCompletion`.

13. How do you implement rate limiting in a Spring application?

- Rate limiting can be implemented using tools like Spring Cloud Gateway with `RateLimiter` filter, or by using third-party libraries like Bucket4j, or implementing custom interceptors to enforce rate limits.

14. What are the key differences between synchronous and asynchronous communication in microservices?

- Synchronous communication requires the client to wait for a response (e.g., REST API), leading to tight coupling and potential delays. Asynchronous communication allows the client to proceed without waiting (e.g., using message queues), improving resilience and scalability.

15. Explain the `Spring HATEOAS` and its use cases.

- Spring HATEOAS (Hypermedia As The Engine Of Application State) is a library that simplifies creating RESTful APIs by providing links to related resources, helping clients navigate the API dynamically.

Questions for 5-10 Years of Experienced Person

1. How do you design a resilient microservices architecture using Spring?

- Designing a resilient microservices architecture involves implementing circuit breakers, retries, fallbacks (using Hystrix or Resilience4j), distributed tracing, service discovery, and load balancing (using Spring Cloud components).

2. How do you implement CQRS (Command Query Responsibility Segregation) in a Spring application?

- Implement CQRS by separating the command and query models. Use Spring Data JPA for the write side and Spring Data Elasticsearch for the read side, ensuring each model is optimized for its purpose.

3. What is Event Sourcing and how do you implement it in Spring?

- Event Sourcing stores the state changes as a sequence of events. Implement it in Spring by using an event store (like Axon Framework), capturing events for each state change, and rebuilding state by replaying events.

4. How do you implement distributed transactions in Spring Boot microservices?

- Implement distributed transactions using the Saga pattern or two-phase commit protocol (2PC). Tools like Spring Cloud Data Flow, Axon Framework, and transactional messaging systems can help manage distributed transactions.

5. Explain the concept of Service Mesh and its role in a microservices architecture.

- A Service Mesh is an infrastructure layer that handles communication between microservices. It provides traffic management, security, observability, and resilience features. Istio is a popular service mesh that can be integrated with Spring microservices.

6. How do you optimize Spring applications for cloud-native deployment?

- Optimize for cloud-native deployment by using 12-factor app principles, containerizing applications (using Docker), deploying to Kubernetes, externalizing configurations, and using Spring Boot and Spring Cloud for seamless cloud integration.

7. What is the role of **Reactive Streams** in Spring WebFlux?

- Reactive Streams in Spring WebFlux provide a standard for asynchronous stream processing with non-blocking backpressure. They enable building scalable and resilient reactive applications.

8. How do you implement blue-green deployment and canary releases in a Spring Boot application?

- Implement blue-green deployment by maintaining two environments (blue and green) and switching traffic between them. Canary releases involve gradually rolling out changes to a small subset of users before a full rollout. Use tools like Kubernetes and Spring Cloud Kubernetes for deployment strategies.

9. What is the significance of `@EnableScheduling` and `@Scheduled` annotations in Spring?

- `@EnableScheduling` enables Spring's scheduling capabilities. `@Scheduled` is used to annotate methods with scheduled tasks, allowing them to run at fixed intervals, cron expressions, or with fixed delays.

10. How do you design a highly available and scalable Spring Boot application?

- Design a highly available and scalable Spring Boot application by using horizontal scaling (load balancers, clustering), stateless service design, externalizing state (using databases or distributed caches), and employing auto-scaling features in cloud environments.

11. Explain the role of Spring Cloud Config in a microservices architecture.

- Spring Cloud Config provides a centralized configuration service for managing configuration properties across multiple microservices, ensuring consistency and ease of management.

JavaScaler